

Pontificia Universidad Católica Madre y Maestra



Presentado por:

Joseph Manuel Ureña Durán

Adrian Alexander Artiles Vargas

ID:

10164781

10170038

Asignatura:

Fundamento de la programación.

Tópico:

Gestión de préstamos de biblioteca.

Maestra:

Damarys Germosén

Fecha de entrega:

25 de junio de 2026

1. Análisis

Para resolver el problema de la gestión de préstamos bibliotecarios, aplicamos los principios del pensamiento computacional enfocándonos fuertemente en la asignación dinámica de memoria y la manipulación manual mediante punteros.

- **Abstracción:** Visualizamos el inventario de la biblioteca y el registro de estudiantes no como variables estáticas, sino como bloques dinámicos de memoria que crecen según la necesidad del usuario. Para lograr esto, abstraímos la entidad "Préstamo" anidándola dentro de la entidad "Estudiante", lo que nos permitió entender que cada estudiante posee su propio sub-arreglo de memoria reservado exclusivamente para sus transacciones.
- **Descomposición:** En lugar de abordar el problema en un solo bloque complejo, dividimos la lógica en un flujo secuencial estricto mediante funciones separadas, cada una con una responsabilidad única y bien definida:

`tam_libros/est`: solicitan y validan exclusivamente los límites del sistema antes de reservar cualquier bloque de memoria. Separarlas de las funciones de registro nos permitió garantizar que `malloc` nunca reciba un valor fuera del rango permitido.

`registrar_libros/est`: Se encargan de poblar los bloques de memoria ya reservados con datos reales del usuario. Cada campo se captura en su propio ciclo `do-while` independiente, lo que nos permitió validar individualmente que ninguna cadena quede vacía y que valores numéricos como cantidad sean mayores a cero. Adicionalmente, `registrar_est` calcula en tiempo real los libros disponibles para limitar dinámicamente la cantidad de préstamos que el estudiante puede solicitar, evitando comprometer stock inexistente desde el inicio.

`registrar_prestamos`: Centraliza toda la lógica del sistema. Primero verifica si hay stock disponible antes de cada préstamo. Luego presenta el catálogo actualizado en tiempo real, valida que el libro elegido tenga copias disponibles, impide que el mismo estudiante seleccione un libro ya registrado en su listado, y descuenta directamente del inventario original mediante $(l+k) \rightarrow \text{cantidad}$ para mantener consistencia entre todos los estudiantes. Además, maneja el escenario donde los libros se agotan a mitad del registro, usando `realloc` para reducir el bloque al tamaño real utilizado y conservando los préstamos ya registrados.

`cantidad`: Implementa la lógica recursiva exigida por la consigna. En cada llamada suma $p \rightarrow \text{cantidad_prestada}$ del préstamo actual y se llama a sí misma con $p+1$ para avanzar al siguiente bloque de memoria, reduciendo el contador `c` en uno hasta alcanzar el caso base $c < 1$ que detiene la recursión y retorna cero. Esto produce la suma acumulada total de copias prestadas por el estudiante.

`mostrar_reporte`: Recorre todos los estudiantes imprimiendo sus datos y préstamos, invocando `cantidad` para obtener el total recursivo de cada uno. Luego genera el reporte de stock restante por libro y calcula el libro más prestado acumulando en un arreglo dinámico `conteo_libros` las copias prestadas de cada título mediante triple recorrido. Finalmente determina el estudiante con

mayor y menor cantidad de libros prestados comparando los totales recursivos de cada uno.

leer_int/limpiar_salto: Funcionan como utilidades transversales reutilizadas en todo el programa. leer_int garantiza que scanf reciba siempre un entero válido limpiando el buffer ante entradas no numéricas. limpiar_salto elimina el '\n' que fgets incluye al final de cada cadena y limpia el buffer cuando el usuario excede el límite de caracteres, previniendo lecturas falsas en iteraciones posteriores.

- **Reconocimiento de Patrones:** Al analizar la lógica de validación, notamos una similitud inherente en cómo tratábamos las entradas de texto. Reconocer este patrón nos permitió crear la función universal limpiar_salto(char *s), la cual reutilizamos en cada fgets para eliminar automáticamente el salto de línea (\n) y limpiar el búfer del sistema si el usuario excedía el límite de caracteres.

2. Ejemplos de Corrida

Caso A.

```
===== REPORTE DE PRESTAMOS =====
ID: 10170038
Nombre: Adrian Artiles
Prestamos:
  Prestamo 1: Programacion en C - Dennis Ritchie, Cantidad: 4
  Prestamo 2: Python - Mark Python, Cantidad: 6
  Prestamo 3: hola mundo en C - Damarys Germosen, Cantidad: 3
Total libros prestados: 13

ID: 10164781
Nombre: Joseph Duran
Prestamos:
  Prestamo 1: Programacion en C - Dennis Ritchie, Cantidad: 2
  Prestamo 2: Python - Mark Python, Cantidad: 1
Total libros prestados: 3

===== REPORTE DE LIBROS =====
Libro 1: Programacion en C - Dennis Ritchie, copias restantes: 0
Libro 2: Python - Mark Python, copias restantes: 0
Libro 3: hola mundo en C - Damarys Germosen, copias restantes: 0

Libro mas prestado: Python

Presione una tecla para continuar . . . |
```

Caso B.

```
Estudiante 1
ID: 10170038
Nombre: Adrian Artiles
Ingrese numero de prestamos (max 5): 1

===Prestamo 1===
Titulo: El quijote
Autor: Miguel de Cervantes
Copias disponibles: 5
Cantidad a prestar: 7
Debe estar entre 1 y 5
Presione una tecla para continuar . . . |
```

```
Estudiante 1
ID: 10170038
Nombre: Adrian Artiles
Ingrese numero de prestamos (max 5): 1

===Prestamo 1===
Titulo: El quijote
Autor: Miguel de Cervantes
Copias disponibles: 5
Cantidad a prestar: 0
Debe estar entre 1 y 5
Presione una tecla para continuar . . . |
```

Caso C.

```
Ingrese la cantidad de libros (min. 2 y max. 10): v f g h  
Debe estar entre 2-10  
Presione una tecla para continuar . . . |
```

```
Libro 1  
Titulo:  
debes digitar algo  
Presione una tecla para continuar . . . |
```

```
Libro 1  
Titulo: Hola Mundo  
Autor: Adrian Artiles  
Cantidad: 0  
debe ser mayor a 0  
Presione una tecla para continuar . . . |
```

3. Conclusiones Tecnológicas

Trabajar la aritmética de punteros nos permitió entender la diferencia abismal entre programar a alto nivel y controlar el hardware a bajo nivel. Utilizar asignación dinámica (malloc, calloc) nos otorga un control real y directo sobre la computadora, permitiéndonos crear programas que solo consumen la memoria RAM estrictamente necesaria.

Este enfoque directo nos obliga a ser sumamente cuidadosos. Al interactuar con estructuras dinámicas anidadas (un arreglo de préstamos dentro de un arreglo de estudiantes), comprendimos que la liberación de memoria debe ser profunda e inversa. No basta con hacer un `free(est)`; si omitimos liberar primero las sub-estructuras con `free((est+i)->prestamos)`, producimos fugas de memoria (Memory Leaks) invisibles pero letales para el rendimiento del sistema operativo. En conclusión, dominar la gestión dinámica nos hace programadores más conscientes de la eficiencia y la seguridad estructural de nuestro software.

4. Transparencia de Autoría (Uso de IA)

Declaramos de forma honesta que durante el desarrollo de este código utilizamos herramientas de Inteligencia Artificial (Gemini) estrictamente como apoyo de depuración (debugging) y no como generación automática de lógica matemática, la cual es de autoría propia. Nuestra interacción con la IA se limitó a comprender las mejores prácticas en C frente a errores de ejecución y gestión de memoria.

Específicamente, utilizamos los siguientes comandos/prompts:

- **Prompt 1:** "¿Por qué al hacer `free(est)` mi programa sigue dejando memoria ocupada si el estudiante tiene préstamos?"
 - *Conceptos aclarados:* La herramienta nos orientó sobre el concepto de Memory Leaks en estructuras anidadas, explicándonos que la memoria debe liberarse de adentro hacia afuera (Deep Free).
- **Prompt 2:** "¿Cómo puedo limpiar la pantalla de la consola sin usar `system(\"cls\")` para cumplir con el estándar C?"
 - *Conceptos aclarados:* Entendimos que `system()` envía comandos directo al OS (Windows) rompiendo la portabilidad del código, y aprendimos a simular la limpieza mediante saltos de línea y formateo estándar ANSI, aunque finalmente optamos por mantener el control visual limpio asumiendo el entorno Windows.

- **Prompt 3:** "¿Cómo liberar solo el espacio no utilizado cuando se agotan los libros en medio del registro de préstamos?"
 - *Conceptos aclarados:* Entendimos el uso de realloc para reducir un bloque existente al tamaño real utilizado, conservando los datos ya registrados y evitando tanto el desperdicio de memoria como la pérdida de información válida.